



Operating System

Lab 13

Fasiha Tariq - 53289

BS-CS 5

Submitted to:

Ma'am Ayesha Ikram

Date: November 16th, 2025

1. What potential problems can occur when multiple threads access the same data without synchronization?

When multiple threads access shared data without synchronization, several issues can arise:

- **Race Condition:**

When two or more threads read and write shared data at the same time, the final result of the execution depends on the unpredictable order of execution.

Example: Two threads incrementing a shared counter may overwrite each other's updates.

- **Data Inconsistency:**

Unsynchronized access can leave data in an invalid or partially updated state.

Example: thread A writes half of a structure and thread B reads it before it done.

- **Lost Updated:**

One thread's update can overwrite another's, resulting in incorrect results.

- **Unexpected Behavior / Crashes:**

If shared memory is modified while another thread is using it, the program may crash or behave unpredictably.

2. What is a deadlock? Explain a situation where it might occur in multithreading.

A deadlock occurs when two or more threads are permanently blocked, each waiting for a resource held by another thread. As a result, none of them can continue execution.

Example of a Deadlock in Multithreading:

Consider two threads T1, T2, and two mutexes, M1 and M2:

- Thread T1 locks M1, then tries to lock M2

- Thread T2 locks M2, then tries to lock M1

Both threads wait indefinitely:

- T1 waits for M2 (held by T2)
- T2 waits for M1 (held by T1)

Since neither will ever release its first lock, both are stuck forever → **deadlock**.

3. How can thread attributes be used to control thread scheduling or stack size?

Thread attributes (using **pthread_attr_t** in POSIX threads) allow you to customize a thread before creating it. Two common attributes you can control are stack size and scheduling behavior.

1. Stack Size:

Each thread has its own stack, and the default size may not always be sufficient, especially for functions using large variables or deep recursion. You can modify the stack size using:

C – Code:

```
pthread_attr_t attr;  
  
pthread_attr_init(&attr);  
  
pthread_attr_setstacksize(&attr, 1024 * 1024);  
  
pthread_create(&thread, &attr, thread_function, NULL);  
  
pthread_attr_destroy(&attr);
```

Useful when the thread needs more or less memory than the default.

2. Scheduling:

Threads attributes can also control how the thread is scheduled by the OS scheduler.

You can modify it by:

- **Scheduling Policy**

SCHED_FIFO → First in First Out real-time scheduling.

SCHED_RR → Round Robin real-time scheduling.

SCHED_OTHER → Default Linux time-sharing policy.

- **Thread priority**

Higher-priority threads get CPU time sooner.

C- Code:

```
pthread_attr_setschedpolicy(&attr, SCHED_FIFO);
```

Modification of thread scheduling ensures critical tasks get more CPU time and manage priority among multiple worker threads.